

Сопровождение и развитие системы PracticeGrading

Левашев Роман Владимирович

Группа: 24.Б44-мм

Кафедра: системного программирования

Научный руководитель: Литвинов Юрий Викторович

Консультант: Мясникова Мария Геннадьевна, АО Нэксайн, инженер-программист

Номер семестра практики: 4

1. Постановка задачи

PracticeGrading применяется при проведении защит учебных практик на кафедре системного программирования СПбГУ. Система хранит сведения о заседаниях и работах студентов, принимает оценки членов комиссии и рассчитывает рекомендуемый результат по принятым на кафедре критериям. В приложении отсутствовало удобное управление учётными записями, а доступ по ссылке не защищал членов комиссии от подмены. Обновление приложения на рабочем сервере выполнялось вручную. **Цель работы** — повысить безопасность системы и автоматизировать выпуск новых версий.

2. Описание предлагаемого решения

Краткий обзор аналогов. В СПбГУ при организации защит ВКР используется **Blackboard Learn**. Система позволяет принимать работы студентов, оценивать их по заданным критериям и сохранять результаты в электронном журнале. Однако для воспроизведения принятого на кафедре процесса потребовалась бы дополнительная адаптация: представление заседания и состава комиссии, независимое выставление оценок несколькими членами комиссии, расчёт результата по кафедральным правилам и формирование документов.

Существуют и другие универсальные системы управления обучением, например **Moodle** и **Canvas**. Они также предоставляют средства для работы с заданиями, критериями и оценками, но ориентированы прежде всего на проведение учебных курсов. Поэтому было предпочтено развитие **PracticeGrading**, где необходимые сущности и правила проведения защит уже представлены.

Blackboard Learn СПбГУ: <https://bb.spbu.ru/>; документация по электронному журналу **Moodle**: https://docs.moodle.org/en/Grader_report; документация по электронному журналу **Canvas**: <https://community.instructure.com/en/kb/articles/660826-how-do-i-use-the-gradebook> (дата обращения: 21.09.2026).

Реализация

Система использует **ASP.NET Core**, **React** и **PostgreSQL**. Создание администратора и смена пароля доступны через интерфейс приложения. Сервер проверяет роль пользователя, а перед изменением учётных данных повторно проверяет его текущий пароль. Новый пароль должен содержать не менее 12 символов; пароли хранятся в виде хешей **BCrypt**.

Для членов комиссии реализованы подтверждаемый и доверенный доступ. В обычном режиме новый участник может присоединиться во время защиты, но получает право выставить оценки только после одобрения администратором или другим подтверждённым членом комиссии этого заседания. Для заранее известного члена комиссии администратор может сформировать персональную ссылку доверенного доступа. После её однократной активации участник входит в заседания без запроса на подтверждение и при необходимости автоматически добавляется в состав комиссии. Для запросов сохраняются состояния ожидания, одобрения, отклонения и отзыва. Одобрить запрос может администратор или уже подтверждённый участник того же заседания. После одобрения сервер выдаёт **JWT**, ограниченный конкретным заседанием. Доверенная ссылка содержит случайный токен; в базе хранится только его **SHA-256**-хеш. Доверенный доступ можно перевыпустить или отозвать. Наличие действующего разрешения проверяется в базе при каждом защищённом запросе, поэтому отзыв действует и для ранее выданного **JWT**.

Для **Continuous Deployment** в **GitHub Actions** настроен сценарий автоматизации (workflow). После отправки изменений в основную ветку он собирает и тестирует проект, публикует Docker-образ серверной части с идентификатором коммита и формирует артефакт со сборкой клиентской части, SQL-миграциями и метаданными выпуска. Развёртывание выполняет установленный на рабочем сервере агент **GitHub Actions** (self-hosted runner), которому разрешён запуск только одной заранее заданной привилегированной команды.

Сценарий проверяет соответствие идентификаторов артефакта и образа, загружает образ и только затем включает режим обслуживания. Перед изменениями он сохраняет текущие версии серверной и клиентской частей и создаёт резервную копию **PostgreSQL** в специальном архивном формате. Корректность копии проверяется командой `pg_restore --list`. Миграции выполняются по порядку в транзакциях; их номера и контрольные суммы записываются в таблицу `__SchemaMigrations`. После запуска нового контейнера выполняется проверка доступности серверной части (liveness check), а каталог клиентской части переключается атомарной заменой символической ссылки. При ошибке обработчик восстанавливает базу из резервной копии, предыдущий Docker-образ и прежнюю ссылку на клиентскую часть. Режим обслуживания снимается только после успешной проверки восстановленной версии.

Дополнительно сервис и таймер **systemd** ежедневно запускают отдельный скрипт резервного копирования. Файловая блокировка не позволяет одновременно запустить несколько экземпляров скрипта. Скрипт создаёт резервную копию базы **PostgreSQL** в формате архива, проверяет её структуру, сохраняет контрольную сумму **SHA-256** и устанавливает права доступа только для владельца. Копии хранятся на сервере 14 дней; устаревшие файлы удаляются лишь после успешного создания новой копии. Внешнее хранилище пока не подключено.

3. Тестирование

Добавлены модульные и интеграционные тесты на **NUnit** и сквозные сценарии на **Playwright**. Проверены создание администратора, смена пароля, подтверждаемый и доверенный вход, добавление в комиссию и отзыв доступа.

Механизм CD испытан в изолированном окружении, повторяющем состав рабочего сервера. Искусственная ошибка после изменения базы, серверной и клиентской частей завершилась полным откатом. Следующий выпуск успешно активировал новую версию. После настройки рабочего сервера CD успешно установил новую версию. Вручную проверены вход, управление учётными записями, оценки и документы. Для первой ежедневной резервной копии проверены контрольная сумма и каталог архива.

4. Заключение

В ходе выполнения работы получены следующие результаты:

- добавлено управление администраторами и их паролями;
- защищён доступ членов комиссии к оцениванию;
- реализовано последовательное применение версионированных миграций;
- подготовлен и проверен сценарий CD с контролем состояния приложения и автоматическим откатом изменяемых компонентов;
- настроены ежедневные резервные копии базы с хранением в течение 14 дней.

Репозиторий проекта: <https://github.com/yurii-litvinov/PracticeGrading> (дата обращения: 21.09.2026).

Изменения: <https://github.com/yurii-litvinov/PracticeGrading/pull/13> (дата обращения: 21.09.2026).

Материалы и техническая документация: <https://github.com/RomanLevashev/practicegrading-defense>, файл pdf/technical-documentation.pdf (дата обращения: 21.09.2026).

Статус: *изменения установлены на рабочем сервере; CD и ежедневное резервное копирование запущены, откат проверен в изолированном окружении.*